Natural Language Processing - Lecture 04

Lecture overview

Lecture 04 establishes the architectural evolution of neural language modeling, progressing from feed-forward networks and word embeddings to recurrent structures and the Transformer. While feed-forward networks process fixed inputs through weighted layers, Recurrent Neural Networks (RNNs) handle variable-length sequences by maintaining a hidden state; however, simple RNNs suffer from vanishing gradients, a limitation addressed by Long Short-Term Memory (LSTM) units that use gating mechanisms to control information retention. The Transformer architecture supersedes these by utilizing attention mechanisms to generate contextualized embeddings where word meanings adapt to surrounding history. This is achieved through multi-head attention, which projects inputs into query, key, and value vectors to calculate relevance scores, while causal masking ensures predictions rely strictly on preceding tokens in a left-to-right generation process.

The lecture further details the training and application of Large Language Models (LLMs), emphasizing that their capability to solve diverse downstream tasks like sentiment analysis, question answering, and summarization emerges from pretraining on massive self-supervised datasets using next-word prediction. This pretraining minimizes cross-entropy loss via teacher forcing, a strategy where ground-truth tokens are fed as inputs at each step to prevent error propagation during training. During inference, temperature sampling adjusts the probability distribution of generated tokens to balance between greedy selection and flexible exploration. Ultimately, the power of LLMs stems from scaling the Transformer architecture with vast parameter counts and data, enabling them to perform complex conditional generation tasks without explicit task-specific training.

Slide 1: faculty of arts

This lecture introduces Neural Language Modeling as the fourth session in the Natural Language Processing course within the Faculty of Arts. The session is led by Tim Van de Cruys and took place on Thursday, 15 October 2026. No additional conceptual content, definitions, or examples were provided for this introductory slide in the available sources.

Slide 2: Reminder

A single artificial neuron computes a weighted sum of its inputs plus a bias, then applies an activation function. Logistic regression uses this pattern to estimate a class probability. A feed-forward network stacks such computations: each layer transforms the previous layer's activations using learned weights and biases, and training adjusts those parameters to reduce prediction error.

Slide 3: Feed-forward

The feed-forward architecture utilizes a multi-layer perceptron with two hidden layers to process input x into output y. This involves computing the first hidden layer h_1 and second hidden layer h_2 using an activation function g, followed by a final linear transformation, where all weights (W) and biases (b) constitute the network parameters θ. The employment of these multiple hidden layers defines the approach as deep learning.

Slide 4: Word embeddings

Word embeddings map vocabulary items to dense vectors; the embedding matrix has one row per vocabulary item and one column per embedding dimension. Training can place words used in similar ways near one another. A static embedding assigns one vector to a word form, so it does not by itself distinguish different senses of an ambiguous word. Contextual models such as Transformers build a different representation for each occurrence using its surrounding words.

Slide 5: Feed-forward network

A feed-forward language model takes a fixed-size window of preceding word embeddings, combines them in a hidden layer, and uses a softmax output to estimate the next word. Training raises the probability assigned to the observed next word. Because the input window is fixed, the model has a limited context; this objective alone does not make it a general summarizer or question-answering system.

Slide 6: Recurrent neural network

Recurrent neural networks address the fixed-input limitation of feedforward architectures by processing varying amounts of language input to predict the next word. This model builds a historical representation by combining the current word input with the hidden layer from the previous timestep using specific weight matrices and an activation function. The resulting hidden state is then passed through a softmax function to generate a probability distribution for the predicted output.

Slide 7: Concrete RNN architectures

A simple RNN applies the same learned transition at each time step. It combines the current input with the previous hidden state, which carries a summary of earlier inputs, then produces a new hidden state for the next step. This lets the model process sequences of varying lengths while reusing the same parameters.

Slide 8: Concrete RNN architectures

Simple RNNs can have difficulty learning from distant time steps because gradients may shrink as they are propagated backward. LSTMs add a cell state and gates that regulate what information is retained, updated, or exposed. These mechanisms can help preserve useful long-range information, but they do not guarantee perfect or unlimited memory.

Slide 9: Concrete RNN architectures

An LSTM is a recurrent model designed to make information flow across many time steps easier to learn. Its cell state carries information forward, while input, forget, and output gates regulate what gets written, retained, and exposed at each step. Gates help with long-range dependencies but do not guarantee that every relevant detail is remembered.

Slide 10: Transformer

The Transformer is the fundamental building block of LLMs, functioning as an attention-based network that converts static word inputs into contextualized embeddings by integrating surrounding information. In left-to-right language modeling, a residual stream progressively refines these representations through successive blocks to capture preceding context, culminating in a final hidden state used by an unembedding matrix to predict the next word. While this decoder-focused approach handles causal prediction, the architecture also supports encoder and encoder-decoder variants that access full sentence contexts for tasks like machine translation.

Slide 11: Transformer

The Transformer uses attention to build contextualized token representations. The original Transformer is an encoder-decoder architecture; many current generative language models use a decoder-only causal variant, while encoder-only and encoder-decoder models remain useful for other tasks. In causal generation, each position predicts a next token using its permitted left context.

Slide 12: Transformer

The example uses context to clarify the word “star”: an astronomer watching the star suggests an astronomical object, while an audience watching the star in awe suggests a performer. A Transformer uses surrounding tokens to form contextualized representations, so the same word form can be represented differently in these two contexts.

Slide 13: Transformer

Slide 13 introduces the Transformer architecture as a key topic within neural language modeling, citing Jurafsky and Martin (2026) as the primary reference. The slide serves as a section header for this specific model, identified by the course instructor Tim Van De Cruys at KU Leuven. No additional technical details, definitions, or examples regarding the Transformer's mechanics are provided in the available source material for this specific slide.

Slide 14: Attention

In causal language modeling, attention operates left-to-right, restricting each token's view to only preceding and current inputs. When processing a specific token x_i, the model attends to all inputs up to and including x_i itself; for instance, x_1 relies solely on itself, while x_3 incorporates information from x_1, x_2, and x_3. This mechanism ensures predictions are based exclusively on available historical context rather than future tokens.

Slide 15: Attention: an intuitive simplification

Attention is presented as an intuitive simplification where the relevance between tokens is calculated via a dot-product score, $score(x_i, x_j) = x_i \cdot x_j$. These scores are normalized into attention weights α_{ij} using softmax over all preceding positions ($j \le i$), and the final context vector a_i is computed as the weighted sum of those previous token representations.

Slide 16: An attention head

An attention head replaces original input vectors with three derivative vectors-query, key, and value-projected via weight matrices W_Q, W_K, and W_V. The query vector for the current element computes a similarity score against preceding key vectors using a dot product, which is normalized to generate weights. These weights determine a weighted average of the corresponding value vectors, creating a contextualized representation that blends relevant information from previous inputs rather than relying on raw vectors alone.

Slide 17: An attention head

For each token, learned projections produce a query, key, and value. The query is compared with keys at the positions the attention mask allows; the resulting weights combine the corresponding values into a context-sensitive output. The phrase in the diagram illustrates that the token representations depend on the sequence and position being processed.

Slide 18: Attention computation

Attention computation projects each input x into a query, key, and value using W_Q, W_K, and W_V. For position i, the query is compared with the allowed keys; each dot product is scaled by √d_k and softmax converts the scores into weights α_ij. The output a_i is the weighted sum of the corresponding values. In a causal language model, the allowed positions include the current and earlier tokens, but exclude future tokens.

Slide 19: Visual representation

This diagram walks through one attention-head calculation: input vectors are projected to queries, keys, and values; a query is scored against the keys; the scores are scaled and passed through softmax; and the resulting weights combine the value vectors into one output. The causal mask limits which keys a position can use during left-to-right prediction.

Slide 20: Multi-head attention

Multi-head attention runs several learned attention projections in parallel. Each head produces an output from its own query, key, and value projections; the outputs are concatenated and passed through an output projection. The model can learn different relation patterns across heads, though an individual head is not guaranteed to have one simple linguistic role.

Slide 21: Multi-head attention: visual representation

The figure expands the multi-head mechanism: separate heads attend to the sequence, their outputs are concatenated, and the concatenated vector is multiplied by the output matrix W^O to return to the model dimension. This projected result is the multi-head attention output that can be added to the residual stream in a Transformer block.

Slide 22: The residual stream

The residual stream is the representation carried through Transformer layers. Sublayers such as attention and feed-forward networks add updates to that stream through residual connections, which preserve a direct information path and help optimization; it is not simply a single vector with a fixed human-readable meaning.

Slide 23: Layer Norm

Layer Norm applies a z-score-like normalization to a single vector, ensuring it has a mean of 0 and standard deviation of 1 by subtracting the computed mean and dividing by the standard deviation. This step prevents vectors from becoming too large, allowing them to be properly added to the residual stream for smooth computation. Finally, learnable parameters gamma and beta are applied to control the scale and shift of the normalized representation.

Slide 24: Feedforward network

The Transformer block incorporates a standard feedforward network with one hidden layer to model non-linearities within representations. This component computes a hidden representation by multiplying the input by a weight matrix, applies a non-linear activation function (originally ReLU), and then multiplies by a second weight matrix to produce the final output. The resulting representation is added back into the residual stream, often following a layer normalization step that ensures computations remain stable and values are appropriately scaled for integration.

Slide 25: Putting it all together

This slide assembles a Transformer block: normalize the current representation, apply multi-head self-attention, and add the result through a residual connection; normalize again, apply the position-wise feed-forward network, and add that result through a second residual connection. In this causal language-modeling setting, attention cannot use future tokens. The block can be repeated while keeping the residual-stream dimension consistent.

Slide 26: Putting it all together

Transformer blocks can be stacked because each block returns a representation with the model dimension d. Multi-head outputs are concatenated and projected back to d, so the next block can accept them and residual additions remain shape-compatible. During training, token positions in a known sequence can be computed in parallel under the causal mask; autoregressive generation still produces tokens one at a time.

Slide 27: Putting it all together

Common modern decoder variants make design choices beyond the basic block shown here. Examples include RMSNorm instead of LayerNorm, gated feed-forward networks, rotary position embeddings applied to queries and keys, and grouped-query attention, where several query heads share key/value heads to reduce inference memory and work. These are common options, not features shared by every current model.

Slide 28: Input: token embeddings plus position embeddings

The model input combines token embeddings from a vocabulary matrix with position embeddings, which are randomly initialized vectors learned during training to encode word order. This addition is essential because the attention mechanism inherently loses positional information, making it impossible to track where words appear in a sentence without this explicit signal. The final input representation is generated by summing the specific token embedding and its corresponding position embedding.

Slide 29: Visual representation

This diagram shows how a token embedding is combined with a position embedding before the sequence enters a Transformer block. Token embeddings carry learned information about token identity; the position signal supplies order information that self-attention would not get from an unordered set alone.

Slide 30: Language modeling head

The language-model head maps the final contextualized representation at the prediction position to vocabulary scores, or logits. A softmax converts those scores into a next-token probability distribution.

Slide 31: From logits to probabilities

Logits represent unnormalized scores for each vocabulary token, theoretically ranging from negative to positive infinity, which are generated via an unembedding layer often transposed from the initial embedding matrix. To convert these raw scores into a valid probability distribution suitable for sampling, the softmax function is applied to normalize them. This process transforms the model's spread-out output distribution into probabilities that can be compared against the true, peaky target distribution where the correct word has a probability of one.

Slide 32: Decoding

Decoding converts the model's output probability distribution into a specific next token using strategies like greedy decoding or sampling. Greedy decoding selects the single most probable word, but this often yields repetitive and uninteresting text. Conversely, random sampling introduces variety by selecting tokens according to their probabilities, yet it risks choosing unlikely words from the distribution's tail. To balance coherence with diversity, researchers employ techniques like top-k and top-p sampling, which restrict selection to likely candidates while maintaining randomness.

Slide 33: Improved decoding: restrict the vocabulary

Improved decoding restricts the vocabulary to avoid sampling from unlikely words in the probability tail. Top-k sampling selects a fixed number of most likely words (e.g., 50) and renormalizes them, though this fixed limit may not suit contexts where grammatical necessity demands high certainty versus open-ended possibilities like adjectives. Top-p (nucleus) sampling addresses this by dynamically selecting the smallest word set covering a specific probability mass percentage (e.g., top 80%), ensuring the vocabulary size adapts to the model's confidence at each position.

Slide 34: Improved decoding: reshape the probability distribution

Temperature rescales logits before softmax. At $\tau = 1$ the distribution is unchanged; lowering τ makes it sharper and more concentrated on high-scoring tokens, while raising τ flattens it and increases the chance of lower-scoring choices. Temperature changes sampling behavior, not the model’s learned weights.

Slide 35: The power of scale

Scaling Transformer models and training data with next-token prediction can produce useful behavior on tasks such as sentiment analysis, question answering, or summarization. These abilities are not guaranteed by the objective alone: performance depends on the model, data, prompt, and any task-specific adaptation.

Slide 36: Conditional generation

This section introduces the concept of conditional generation within neural language modeling, referencing the framework established by Jurafsky and Martin (2026). As no specific definitions, examples, or clarifications were provided in the aligned transcript for this slide, the material serves solely as a structural heading to denote the upcoming topic.

Slide 37: Examples

Neural language models solve diverse tasks like sentiment analysis and question answering by predicting the next most probable word token-by-token. For instance, given the prompt "The sentiment of the sentence 'The new Bond movie is fantastic' is:", the model generates the sentiment label as the next word. Similarly, for "The capital of France is...", it predicts the correct city name. This capability emerges from applying the next-word prediction training objective at an enormous scale, allowing the model to provide direct answers to these queries.

Slide 38: Summarization

This slide illustrates summarization by contrasting a detailed news story about entrepreneur Kyle Waring selling Boston snow with a condensed version that retains only essential facts. The original text describes the $89 price, 6-pound quantity, shipping restrictions to New England, and the origin of the idea during record-breaking snowfall. In contrast, the provided summary extracts just the core transaction details and geographic limitations, demonstrating how summarization reduces length while preserving the primary message.

Slide 39: Summarization

Slide 39 introduces the topic of summarization within the context of neural language modeling, referencing Jurafsky and Martin (2026) as the primary source material. This section serves as a structural marker in the lecture series, identified by the course email tim.vandecruys@kuleuven.be and positioned as slide 37 of 53 in the referenced text. No specific definitions, examples, or technical clarifications regarding summarization methods are provided in the available sources for this particular slide.

Slide 40: Pretraining with self-supervision

Pretraining is the initial step for Large Language Models, utilizing vast web-extracted datasets like CommonCrawl or curated versions such as FineWeb and Dolma, often totaling 10-15 trillion tokens. This process is termed "self-supervised" because it relies on plain text without requiring manual annotations; instead, the model learns by predicting the next word in a sequence, using the subsequent token itself as the training label.

Slide 41: Objective function: cross-entropy loss

The cross-entropy loss function aims to assign high probability to the true next word by utilizing its negative log probability as the loss metric. Consequently, the model incurs a large loss when assigning low probability to the correct word and a small loss when the prediction is accurate. During training, this loss is computed across multiple positions simultaneously, driving gradient-based updates to attention and feed-forward weight matrices that minimize the overall error.

Slide 42: Teacher forcing

Teacher forcing is a training strategy where, at each token position t, the model receives the correct ground-truth tokens $w_{1:t}$ as input to compute the loss for predicting the next token w_{t+1}. Crucially, even if the model predicts an incorrect word, its prediction is ignored for the subsequent step; instead, the actual true word from the corpus is fed into the context for position $t+1$. This ensures that the network always bases its next prediction on the correct historical sequence rather than its own potentially erroneous outputs.

Slide 43: Visual representation

The diagram summarizes next-token training: input tokens pass through Transformer blocks and a language-modeling head to produce vocabulary logits at each position. The target at each position is the following token, and cross-entropy loss measures the probability assigned to those targets. A causal mask prevents the prediction at one position from using later tokens.

Slide 44: Finetuning

Slide 44 introduces the topic of finetuning within the context of neural language modeling, referencing Jurafsky and Martin (2026) as the primary source material. As no specific definitions, examples, or clarifications regarding the mechanics or rationale of finetuning are provided in the aligned transcript or slide text, this section serves solely as a structural heading indicating the upcoming discussion on adapting pre-trained models.

Slide 45: Post-training

Post-training adapts a pretrained model toward assistant behavior using methods such as instruction tuning and preference optimization. The slide names approaches including RLHF/PPO and DPO, as well as LoRA for parameter-efficient adaptation. These methods can steer outputs toward preferred behavior, but they do not guarantee safety or accuracy.

Slide 46: The downside of LLMs

The slide lists risks including hallucination, copyright disputes, privacy, bias, toxicity, misinformation, and environmental cost. Next-token training does not itself establish whether an output is true. Training on copyrighted works raises ongoing questions about permission and compensation; handling user data also creates privacy concerns.

Slide 47: Three architectures for LLMs

The slide diagrams three common Transformer architectures: decoder-only models use causal attention for generation; encoder-only models use bidirectional context and are often adapted to classification or token labeling; encoder-decoder models transform an input sequence into an output sequence, as in translation. These are common patterns rather than an exhaustive taxonomy.

Slide 48: Three architectures for LLMs

The examples illustrate the same three layouts with their typical uses: decoder-only models predict the next token, encoder-only models build representations from both left and right context, and encoder-decoder models map one sequence to another. A task may still require prompting, fine-tuning, or a task-specific output head.

Slide 49: Bi-directional attention

The figure contrasts attention masks. Causal self-attention lets a token use itself and earlier positions, blocking future positions so next-token training matches generation. Bidirectional self-attention lets a token use context on both sides; it is useful for understanding a complete input, but ordinary next-token training would leak the answer from the visible future.

Slide 50: Masked language modeling

With bidirectional attention, the model can see tokens on both sides, so the pretraining task hides or corrupts selected tokens and asks the model to reconstruct them. The loss is applied at the selected positions. The diagram illustrates why masked language modeling supplies a prediction target without letting the model simply read the original token from its context.

Slide 51: Next sentence prediction

Next-sentence prediction is a BERT pretraining task that classifies whether a second sentence follows the first in the source text. It is a sentence-pair classification objective; it does not require the model to generate the next sentence.

Slide 52: Contextualized embeddings

BERT-like encoder models excel at creating contextualized embeddings by processing entire sentences as input, granting access to both left-hand history and right-hand future context. Unlike causal language modeling in decoders which only sees the past, encoders utilize attention mechanisms across all tokens to capture bidirectional meaning. This comprehensive context allows these models to generate rich representations suitable for fine-tuning on specific classification tasks.

Slide 53: Fine-tuning for classification

Fine-tuning for classification adapts a pre-trained encoder, originally trained on large text corpora via masked language modeling, to specific tasks by adding an appropriate classification head and training on curated, hand-labeled data. While pre-trained parameters can be frozen or minimally adjusted, the model is pushed toward the target task using examples like sentiment analysis or grammatical error correction. This process leverages self-supervised pre-training to efficiently solve downstream problems with binary or multi-class outputs calculated via softmax.

Slide 54: Sequence classification

For sequence classification, the whole input is assigned one label. In the BERT setup shown, the final representation for the special [CLS] token is passed to a classification head; for example, the head can predict a sentiment label for a review.

Slide 55: Sequence labeling

Sequence labeling assigns a label to each token in a sequence, as in part-of-speech tagging or named-entity recognition. This differs from sentence classification, which assigns one label to the whole input.

Slide 56: Different classification configurations

The diagram shows four ways to use an encoder for labeled tasks: classify a whole sequence from its pooled or [CLS] representation; label each token; predict answer start and end positions for question answering; or classify a pair of sentences. The task head changes to match what must be predicted, while the encoder supplies contextualized token representations.

Key points to remember

The lecture further details the training and application of Large Language Models (LLMs), emphasizing that their capability to solve diverse downstream tasks like sentiment analysis, question answering, and summarization emerges from pretraining on massive self-supervised datasets using next-word prediction. This pretraining minimizes cross-entropy loss via teacher forcing, a strategy where ground-truth tokens are fed as inputs at each step to prevent error propagation during training. During inference, temperature sampling adjusts the probability distribution of generated tokens to balance between greedy selection and flexible exploration. Ultimately, the power of LLMs stems from scaling the Transformer architecture with vast parameter counts and data, enabling them to perform complex conditional generation tasks without explicit task-specific training.